



LÝ THUYẾT RAG

Retrieval-Augmented Generation

Cơ chế giúp LLM trả lời dựa trên tri thức được truy xuất, có nguồn và có thể kiểm chứng



VÌ SAO CẦN RAG?

RAG giải quyết 3 giới hạn lớn của LLM

01



**Tri thức không
cập nhật**

02



**Thiếu nguồn
kiểm chứng**

03



**Hallucination
nghe hợp lý nhưng sai**

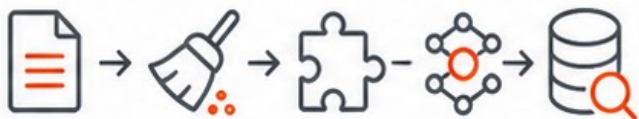


RAG chuyển trọng tâm từ mô hình nghĩ gì sang tài liệu nói gì.

Một pipeline RAG gồm hai luồng chính

01

Offline: Ingest →
Clean → Chunk →
Embed → Index



02

Online: Query →
Retrieve → Rerank →
Generate → Validate



03

RAG tốt bắt đầu từ
pipeline dữ liệu tốt



Vector DB chỉ là lớp index của Knowledge Base



Inventory: doc_id, version, status



Clean data: tài liệu chuẩn hóa và giữ cấu trúc



Metadata: section, source, domain, access_level, chunk_id



section



source



domain



access_level



chunk_id



Knowledge Base tốt cần quản trị tài liệu, metadata, versioning và phân quyền.

Hai kỹ thuật phục vụ hai mục tiêu khác nhau

RAG



Tri thức thay đổi



Cần citation và audit



Kiểm soát version và quyền truy cập

Fine-tuning



Điều chỉnh style và format



Học task pattern ổn định



Không phù hợp để nạp tri thức thay đổi liên tục



Với tri thức nội bộ thay đổi thường xuyên, **RAG thường an toàn hơn.**

CHUNKING

Chunking quyết định context mà LLM nhìn thấy

01



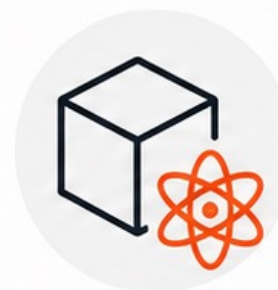
Fixed-size: dễ làm
nhưng dễ cắt đứt
ý nghĩa

02



Semantic: giữ mạch
chủ đề và cấu trúc
tài liệu

03



Proposition: mỗi
chunk là một sự kiện
nguyên tử



Chunk sai tạo context sai; context sai dẫn đến hallucination.

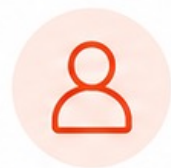
Metadata là lớp lọc trước khi tìm vector



Metadata giúp **truy ngược nguồn, lọc đúng phạm vi và kiểm soát quyền.**

BM25 và Vector Search bù điểm mù cho nhau

BM25



Tên riêng



Mã số



Thuật ngữ chính xác

Vector Search



Diễn đạt tự nhiên



Đồng nghĩa



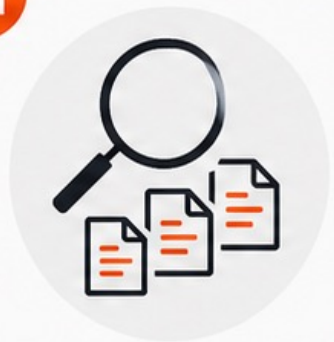
Tìm theo ý nghĩa



Hybrid Search **tăng độ bao phủ** trước reranking.

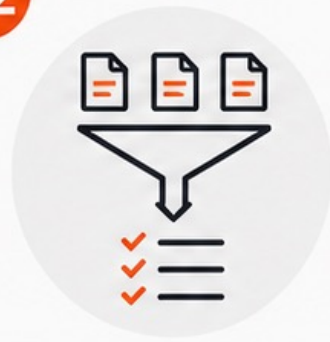
Tăng chất lượng context trước khi gọi LLM

01



Retriever lấy
nhiều ứng viên

02



Cross-encoder
chọn top-k
chính xác hơn

03



HyDE giảm
khoảng cách
thuật ngữ

04



Bằng chứng vẫn
phải đến từ
tài liệu thật

RAG đáng tin phải biết khi nào không trả lời

✓ CHECKLIST

- ✓ Score thấp: `out_of_scope`
- ✓ Câu hỏi mơ hồ: `needs_clarification`
- ✓ Prompt injection: chặn trước generation



Agent đáng tin biết giới hạn của mình.

Câu trả lời RAG cần kiểm được bằng máy

terminal / json / log		
status	:	✓ ok / partial / out_of_scope
answer	:	🗨️ Câu trả lời cuối cùng của hệ thống
doc_id	:	📄 ID của tài liệu nguồn
section	:	☰ Phần / tiêu đề trong tài liệu
chunk_id	:	# ID của chunk được truy xuất
verbatim_quote	:	“” Đoạn trích nguyên văn làm bằng chứng
confidence	:	📊 Điểm tin cậy (0.0 – 1.0)
needs_human_review	:	⚠️ true / false



Quote không khớp phải đánh dấu **rủi ro hallucination**.

EVALUATION

Không đo lường thì không thể cải thiện RAG



Retrieval

hit@k, MRR,
recall@k, nDCG



Generation

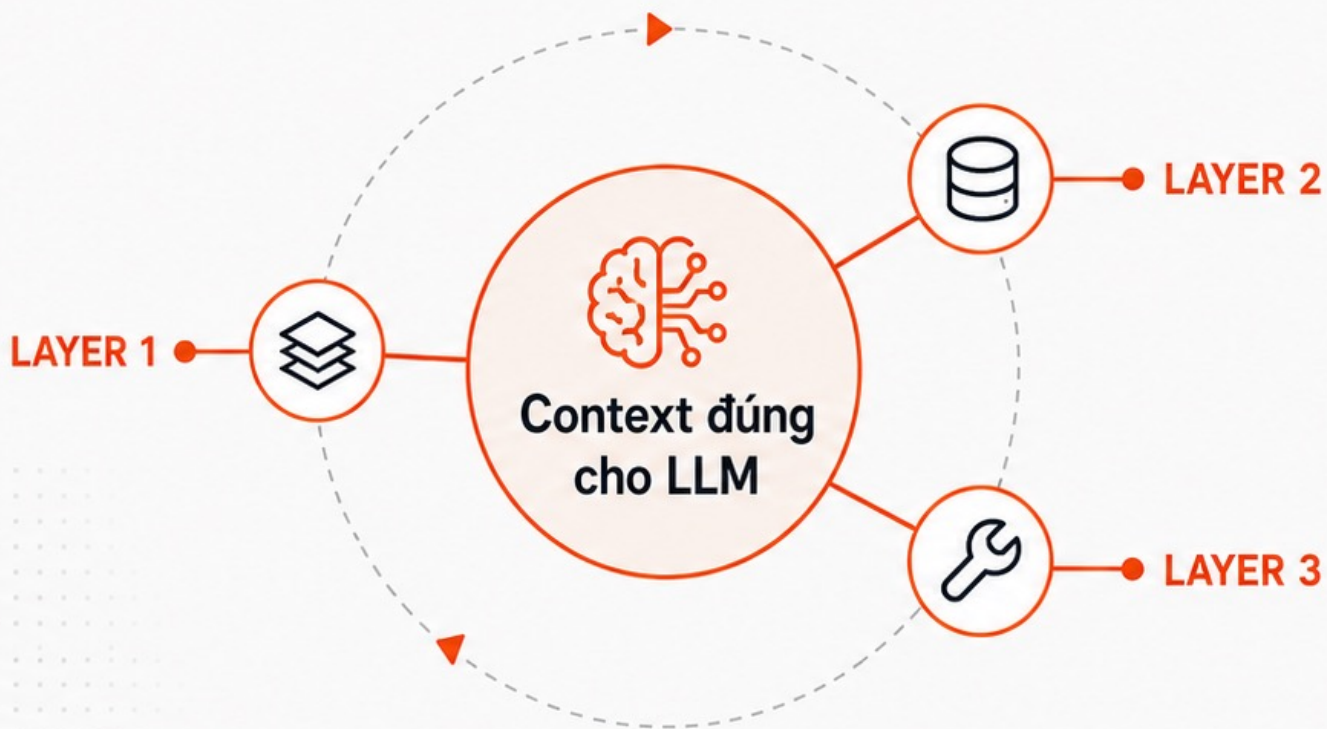
faithfulness, relevancy,
citation accuracy



Quality gates

refusal đúng và
quote khớp nguồn

RAG là một phần của Context Engineering



01

RAG: truy xuất tri thức đã index

02

Memory: lưu ngữ cảnh và thông tin đã xác minh

03

Tool Use: gọi API, database, search hoặc workflow



Dùng đúng lớp context cho **đúng loại câu hỏi.**

Agentic RAG thêm lập kế hoạch và tự sửa lỗi

01



Planner tách câu hỏi
thành sub-query

02



Grader lọc chunk
không liên quan

03



Critic kiểm tra
câu trả lời trước khi trả về



| Phù hợp với câu hỏi **multi-hop** và cần **tổng hợp nhiều nguồn**.



KẾT LUẬN

RAG đáng tin cậy là RAG được engineering đúng

Đúng context + đúng bằng chứng + đúng kiểm chứng

